CENTRAL BOARD OF SECONDARY EDUCATION

JAWAHAR NAVODAYA VIDYALAYA

TARIKHET ALMORA UTTARAKHAND 263663

COMPUTER SCIENCE (083)

PROJECT FILE

On

BANK ACCOUNT MANAGEMENT SYSTEM USING BINARY FILE

Session: 2026-27

Submitted By:
Name: _____
Class: XII Science
CBSE Roll No: _____

Submitted To:
Pankaj Bhilingwal
PGT Computer Science
J.N.V Tarikhet Almora

ACKNOWLEDGEMENT

I wish to express my deep sense of gratitude and indebtedness to our learned teacher Pankaj Bhilingwal, PGT Computer Science, J.N.V Tarikhet Almora for his invaluable help, advice and guidance in the preparation of this project file.

I am also greatly indebted to our principal D.S Rawat and school authorities for providing me with the facilities and requisite laboratory conditions for making this project file.

I also extend my thanks to a number of teachers, my classmates and friends who helped me to complete this project file successfully.

Signature of Student

CERTIFICATE

This is to certify that,

S. No	Name	CBSE Roll

Student of Class XII, J.N.V Tarikhet Almora has completed the PROJECT FILE titled “Bank Account Management System Using Binary File” during the academic year 2026-27 towards partial fulfilment of credit for the Computer Science Project evaluation of CBSE and submitted satisfactory report, as compiled in the following pages, under my supervision.

Internal Examiner

Signature

External Examiner

Signature

PREFACE

In the field of Computer Science, the term PROJECT refers to a computerization work that is completely performed by using a computer. Today almost every field is computerized, and the different tasks of every field are done using computers and appropriate computer programs. Such programs and software are developed in the form of Projects.

Learning Python in schools is a part of our syllabus, and according to the syllabus worked out by the CBSE, to which my school is affiliated, I enjoyed doing project work for the session 2026-27. I learned a lot, especially learning by doing was a great experience. The report is a summary of what I observed and learnt in school. My humble thanks go to all those concerned with my project.

I also feel that it is the right time to thank my teacher, who guided me during my project, and my parents, friends and project partners for the constant assistance and cooperation extended by them from time to time.

INDEX

S. No	Topic	Page No
1	About Project	6
2	Syllabus Concepts Used	8
3	Flow Chart	9
4	Project Code	10
5	Output	14
6	Technological Requirements: Hardware & Software	20
7	References	20

About Project

Bank Account Management System Using Binary File

The Bank Account Management System is a menu-driven Python program that works like a small bank. It can open new accounts, deposit and withdraw money, transfer money between accounts and show the balance of any account.

Account records are stored as dictionaries in a binary file (accounts.dat) using the pickle module. The program uses exception handling so that wrong inputs, missing accounts or low balance never crash the program. It also has an UNDO feature that reverses the last transaction using a stack.

Project Overview

Objective:

The objective of this project is to practise binary file handling and exception handling together in a real-life application, and to show how a stack can be used to provide an Undo facility, as seen in many software applications.

Key Features:

1. **Create Account:** Opens a new Saving or Current account with an automatically generated account number (1001, 1002, ...). Minimum opening amount is Rs. 500.
2. **Deposit Money:** Adds money to an account and saves the updated balance in the file.
3. **Withdraw Money:** Withdraws money only if a minimum balance of Rs. 500 remains.
4. **Transfer Money:** Moves money from one account to another account.
5. **Balance Enquiry:** Shows the details and balance of one account.
6. **Display All Accounts:** Shows all accounts in a neat table.
7. **Undo Last Transaction:** Reverses the latest deposit, withdrawal or transfer using a stack (LIFO).
8. **Error Handling:** Handles wrong input, account not found and insufficient balance with clear messages.

Technology Stack

Programming Language: Python 3

Modules Used: pickle

Each account is a dictionary {acc_no, name, type, balance}. New records are added with pickle.dump() in append mode ('ab'). To read all records, pickle.load() is called in a loop until EOFError is raised. To update a record, all records are read into a list, changed, and written back with mode 'wb'. A list named transactions works as a stack for the Undo feature.

Files in this Project

File	Purpose
bank.py	Main program
accounts.dat	Binary file storing the account records (created automatically)
flowchart.py	Python program that draws the flowchart using graphviz

flowchart.png	Flowchart image
img/	Screenshots of the output

How to Run the Program

1. Make sure Python 3 is installed on the computer.
2. Copy the project folder to the computer.
3. Open bank.py in IDLE and press F5 (or run: python bank.py).
4. Delete accounts.dat if you want to start with no accounts.

Syllabus Concepts Used

This project covers the following topics of the CBSE Class XII Computer Science (083) syllabus:

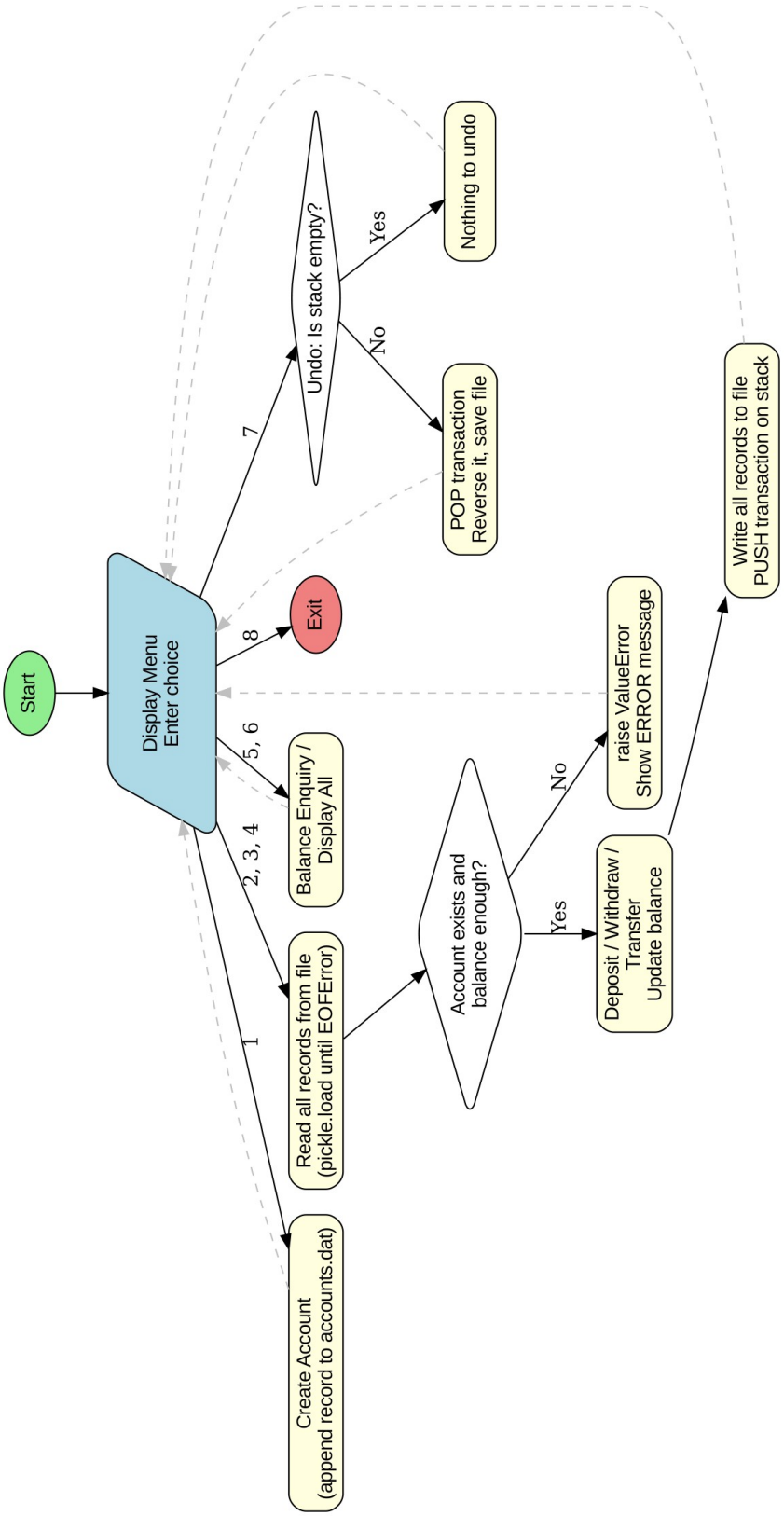
Concept	Where it is used in the project
Binary file handling	pickle.dump() and pickle.load(); file modes 'rb', 'wb' and 'ab'
Reading till end of file	while True loop with pickle.load() stopped by EOFError
Exception handling	try, except ValueError, except FileNotFoundError, except EOFError, finally
Raising exceptions	raise ValueError(...) for invalid account and insufficient balance
Stack	List transactions: append() to PUSH, pop() to POP in undo_last()
Dictionaries	Each account record is a dictionary
User-defined functions	Separate function for each operation, helper functions get_number() and get_account()
Global variables	transactions, FILE_NAME and MIN_BALANCE are used by many functions
String formatting	% formatting to print the accounts table

Why exception handling?

An exception is an error that occurs while the program is running, for example entering 'abc' where a number is expected (ValueError) or opening a file that does not exist (FileNotFoundError). Without handling, the program stops suddenly.

With try-except blocks the program catches the error and shows a friendly message instead. The finally block always runs, so it is used to close the file. The raise statement lets us create our own error, e.g. when the balance is not enough.

Flow Chart


Start
Display Menu
Enter choice
1
Create Account
(append record to accounts.dat)
2, 3, 4
Read all records from file
(pickle.load until EOFError)
Account exists and
balance enough?
No
raise ValueError
Show ERROR message
Yes
Deposit / Withdraw /
Transfer
Update balance
Write all records to file
PUSH transaction on stack
5, 6
Balance Enquiry /
Display All
7
Undo: Is stack empty?
Yes
Nothing to undo
No
POP transaction
Reverse it, save file
8
Exit

Project Code

```
# Bank Account Management System
# Uses: Binary file (pickle), Exception handling, Stack (for UNDO)

import pickle

FILE_NAME = "accounts.dat"
MIN_BALANCE = 500

# Stack to remember transactions of this session (for UNDO)
# Each item: [type, account_no, amount, to_account]
transactions = []


# ---------------- File handling ----------------

def read_all_accounts():
    # Read every record (dictionary) from the binary file
    accounts = []
    f = None
    try:
        f = open(FILE_NAME, "rb")
        while True:
            acc = pickle.load(f)
            accounts.append(acc)
    except FileNotFoundError:
        pass                    # first run: file does not exist yet
    except EOFError:
        pass                    # end of file reached
    finally:
        if f is not None:
            f.close()
    return accounts


def write_all_accounts(accounts):
    f = open(FILE_NAME, "wb")
    for acc in accounts:
        pickle.dump(acc, f)
    f.close()


def find_account(accounts, acc_no):
    for acc in accounts:
        if acc["acc_no"] == acc_no:
            return acc
    return None


# ---------------- Input helpers ----------------

def get_number(message):
    # Keep asking until a valid whole number is entered
    while True:
        try:
            num = int(input(message))
            if num <= 0:
                print("Please enter a number greater than 0.")
            else:
                return num
        except ValueError:
            print("Invalid input! Please enter digits only.")


def get_account(accounts, message):
    acc_no = get_number(message)
    acc = find_account(accounts, acc_no)
    if acc is None:
        raise ValueError("Account " + str(acc_no) + " does not exist.")
```

```
    return acc


# ---------------- Menu options ----------------

def create_account():
    accounts = read_all_accounts()
    if len(accounts) == 0:
        acc_no = 1001
    else:
        acc_no = accounts[-1]["acc_no"] + 1

    name = input("Enter Account Holder Name: ")
    acc_type = input("Enter Account Type (S-Saving / C-Current): ").upper()
    if acc_type == "C":
        acc_type = "Current"
    else:
        acc_type = "Saving"
    amount = get_number("Enter Opening Amount (minimum Rs. 500): ")
    if amount < MIN_BALANCE:
        raise ValueError("Opening amount must be at least Rs. 500.")

    acc = {"acc_no": acc_no, "name": name, "type": acc_type, "balance": amount}
    f = open(FILE_NAME, "ab")    # append a new record
    pickle.dump(acc, f)
    f.close()
    print("Account created successfully! Your Account Number is", acc_no)


def deposit():
    accounts = read_all_accounts()
    acc = get_account(accounts, "Enter Account Number: ")
    amount = get_number("Enter amount to deposit: ")
    acc["balance"] = acc["balance"] + amount
    write_all_accounts(accounts)
    transactions.append(["DEPOSIT", acc["acc_no"], amount, 0])
    print("Rs.", amount, "deposited. New Balance: Rs.", acc["balance"])


def withdraw():
    accounts = read_all_accounts()
    acc = get_account(accounts, "Enter Account Number: ")
    amount = get_number("Enter amount to withdraw: ")
    if acc["balance"] - amount < MIN_BALANCE:
        raise ValueError("Insufficient balance! Minimum balance Rs. 500 must
remain.")
    acc["balance"] = acc["balance"] - amount
    write_all_accounts(accounts)
    transactions.append(["WITHDRAW", acc["acc_no"], amount, 0])
    print("Rs.", amount, "withdrawn. New Balance: Rs.", acc["balance"])


def transfer():
    accounts = read_all_accounts()
    sender = get_account(accounts, "Enter YOUR Account Number: ")
    receiver = get_account(accounts, "Enter RECEIVER Account Number: ")
    if sender["acc_no"] == receiver["acc_no"]:
        raise ValueError("Sender and receiver cannot be the same.")
    amount = get_number("Enter amount to transfer: ")
    if sender["balance"] - amount < MIN_BALANCE:
        raise ValueError("Insufficient balance! Minimum balance Rs. 500 must
remain.")
    sender["balance"] = sender["balance"] - amount
    receiver["balance"] = receiver["balance"] + amount
    write_all_accounts(accounts)
    transactions.append(["TRANSFER", sender["acc_no"], amount, receiver["acc_no"]])
    print("Rs.", amount, "transferred from", sender["acc_no"], "to",
receiver["acc_no"])


def balance_enquiry():
```

```
    accounts = read_all_accounts()
    acc = get_account(accounts, "Enter Account Number: ")
    print("Account No :", acc["acc_no"])
    print("Name       :", acc["name"])
    print("Type       :", acc["type"])
    print("Balance    : Rs.", acc["balance"])


def display_all():
    accounts = read_all_accounts()
    if len(accounts) == 0:
        print("No accounts found.")
        return
    print("%-8s %-20s %-8s %10s" % ("Acc No", "Name", "Type", "Balance"))
    print("-" * 50)
    for acc in accounts:
        print("%-8d %-20s %-8s %10d" % (acc["acc_no"], acc["name"], acc["type"],
acc["balance"]))


def undo_last():
    if len(transactions) == 0:
        print("Nothing to undo! (Stack is empty)")
        return
    last = transactions.pop()        # POP from stack
    kind, acc_no, amount, to_acc = last
    accounts = read_all_accounts()
    acc = find_account(accounts, acc_no)

    if kind == "DEPOSIT":
        acc["balance"] = acc["balance"] - amount
    elif kind == "WITHDRAW":
        acc["balance"] = acc["balance"] + amount
    elif kind == "TRANSFER":
        receiver = find_account(accounts, to_acc)
        acc["balance"] = acc["balance"] + amount
        receiver["balance"] = receiver["balance"] - amount
    write_all_accounts(accounts)
    print("Undo successful:", kind, "of Rs.", amount, "on account", acc_no,
"reversed.")


def main():
    while True:
        print("\n======= BANK MANAGEMENT SYSTEM =======")
        print("1. Create New Account")
        print("2. Deposit Money")
        print("3. Withdraw Money")
        print("4. Transfer Money")
        print("5. Balance Enquiry")
        print("6. Display All Accounts")
        print("7. Undo Last Transaction")
        print("8. Exit")
        choice = input("Enter your choice: ")
        try:
            if choice == "1":
                create_account()
            elif choice == "2":
                deposit()
            elif choice == "3":
                withdraw()
            elif choice == "4":
                transfer()
            elif choice == "5":
                balance_enquiry()
            elif choice == "6":
                display_all()
            elif choice == "7":
                undo_last()
            elif choice == "8":
                print("Thank you for banking with us!")
```

```
                break
            else:
                print("Invalid choice! Please try again.")
        except ValueError as error:
            print("ERROR:", error)


main()
```

Output

Output 1: Creating Accounts

Python Shell - Creating Accounts

```
======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 1
Enter Account Holder Name: Ramesh Negi
Enter Account Type (S-Saving / C-Current): S
Enter Opening Amount (minimum Rs. 500): 5000
Account created successfully! Your Account Number is 1001

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 1
Enter Account Holder Name: Sunita Rawat
Enter Account Type (S-Saving / C-Current): C
Enter Opening Amount (minimum Rs. 500): 12000
Account created successfully! Your Account Number is 1002

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 1
Enter Account Holder Name: Amit Kumar
Enter Account Type (S-Saving / C-Current): S
Enter Opening Amount (minimum Rs. 500): 2500
Account created successfully! Your Account Number is 1003
```

Output 2: Deposit and Withdraw

Python Shell - Deposit and Withdraw

```
======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 2
Enter Account Number: 1001
Enter amount to deposit: 1500
Rs. 1500 deposited. New Balance: Rs. 6500

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 3
Enter Account Number: 1003
Enter amount to withdraw: 800
Rs. 800 withdrawn. New Balance: Rs. 1700
```

Output 3: Exception Handling

Python Shell - Exception Handling

```
======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 3
Enter Account Number: 1003
Enter amount to withdraw: 2000
ERROR: Insufficient balance! Minimum balance Rs. 500 must remain.

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 2
Enter Account Number: abc
Invalid input! Please enter digits only.
Enter Account Number: 1009
ERROR: Account 1009 does not exist.

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 1
Enter Account Holder Name: Test User
Enter Account Type (S-Saving / C-Current): S
Enter Opening Amount (minimum Rs. 500): 200
ERROR: Opening amount must be at least Rs. 500.
```

Output 4: Transfer and Balance Enquiry

Python Shell - Transfer and Balance Enquiry

```
======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 4
Enter YOUR Account Number: 1002
Enter RECEIVER Account Number: 1001
Enter amount to transfer: 3000
Rs. 3000 transferred from 1002 to 1001

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 5
Enter Account Number: 1001
Account No : 1001
Name       : Ramesh Negi
Type       : Saving
Balance    : Rs. 9500
```

Output 5: Display All and Undo

Python Shell - Display All and Undo

```
======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 6
Acc No   Name                 Type        Balance
-------------------------------------------------
1001     Ramesh Negi          Saving         9500
1002     Sunita Rawat         Current        9000
1003     Amit Kumar           Saving         1700

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 7
Undo successful: TRANSFER of Rs. 3000 on account 1002 reversed.

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 6
Acc No   Name                 Type        Balance
-------------------------------------------------
1001     Ramesh Negi          Saving         6500
1002     Sunita Rawat         Current       12000
1003     Amit Kumar           Saving         1700
```

Output 6: Undo Stack Empty and Exit

Python Shell - Undo Stack Empty and Exit

```
======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 7
Undo successful: WITHDRAW of Rs. 800 on account 1003 reversed.

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 7
Undo successful: DEPOSIT of Rs. 1500 on account 1001 reversed.

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 7
Nothing to undo! (Stack is empty)

======= BANK MANAGEMENT SYSTEM =======
1. Create New Account
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Balance Enquiry
6. Display All Accounts
7. Undo Last Transaction
8. Exit
Enter your choice: 8
Thank you for banking with us!
```

Technological Requirements: Hardware & Software

Software

- Python 3.x (IDLE)
- VS Code / any text editor
- Windows 10/11 or Linux

Hardware

- Laptop / PC with Intel Core i3 or above
- 4 GB RAM or more
- Keyboard and Mouse

References

1. Computer Science with Python - Textbook for Class XII (Sumita Arora)
2. NCERT Computer Science Textbook for Class XII, Chapters: Exception Handling, File Handling, Stack
3. https://docs.python.org/3/tutorial/errors.html
4. https://docs.python.org/3/library/pickle.html
5. https://www.python.org/